

**Relatório – Disciplina SD212**  
**Atividade 25-05-2026**  
**Exercício 2 — Cache com Contadores de Hit e Miss**

Este relatório documenta a implementação e simulação de uma cache mapeada diretamente com contadores automáticos de hits e misses, desenvolvida como segunda atividade prática da disciplina de Organização e Arquitetura de Computadores (SD212). O exercício amplia os conceitos do Exercício 1, incluindo uma sequência maior de acessos, substituição por conflito entre endereços distintos e métricas de desempenho quantificadas.

A adição dos contadores `hit_count` e `miss_count` permite calcular automaticamente a taxa de acerto da cache ao final da simulação, tornando o módulo capaz de fornecer informações de desempenho em tempo real. A sequência de 9 acessos foi projetada para demonstrar localidade temporal, conflitos de mapeamento e substituição de blocos.

- **Especificação do Sistema**

A Tabela 1 apresenta os parâmetros que definem a organização da cache implementada neste exercício.

**Tabela 1 — Parâmetros da cache**

| Parâmetro                        | Valor               |
|----------------------------------|---------------------|
| Tipo de cache                    | Mapeada diretamente |
| Número de blocos                 | 8                   |
| Tamanho do bloco                 | 1 palavra           |
| Largura do endereço              | 5 bits              |
| Largura do dado                  | 8 bits              |
| Bits de índice                   | 3 bits (bits [2:0]) |
| Bits de tag                      | 2 bits (bits [4:3]) |
| Contador <code>hit_count</code>  | 8 bits              |
| Contador <code>miss_count</code> | 8 bits              |

- **Divisão do Endereço**

Como o endereço possui 5 bits e a cache possui 8 blocos ( $\log_2(8) = 3$  bits de índice), a divisão é feita da seguinte forma:

$$\text{Endereço}[4:0] = \text{TAG}[4:3] \mid \text{ÍNDICE}[2:0]$$

Exemplo prático: o endereço 10110 é decomposto em tag = 10 (bits [4:3]) e índice = 110 (bits [2:0]), indicando que o bloco deve ser armazenado na posição 6 da cache com tag identificadora 10.

- **Implementação do Endereço**

O módulo cache\_contadores foi implementado com a seguinte interface, estendendo o Exercício 1 com dois contadores de 8 bits:

**Entradas:**

1. clk — sinal de clock do circuito
2. reset — reinicializa cache e contadores
3. read — habilita operação de leitura
4. address[4:0] — endereço de memória de 5 bits
5. mem\_data[7:0] — dado vindo da memória principal em caso de miss

**Saídas:**

6. cache\_data[7:0] — dado retornado pela cache
7. hit — sinaliza acerto na cache
8. miss — sinaliza falha na cache
9. hit\_count[7:0] — contador acumulado de hits
10. miss\_count[7:0] — contador acumulado de misses

- **Estruturas Internas**

O módulo mantém três vetores com oito posições cada, indexados pelo campo índice do endereço, além dos dois contadores:

11. valid[0:7] — vetor de bits de validade (8 posições); zerado no reset
12. tag\_array[0:7] — vetor de tags (2 bits cada) armazenadas na cache
13. data\_array[0:7] — vetor de dados (8 bits cada) correspondentes
14. hit\_count[7:0] — incrementado a cada acerto detectado
15. miss\_count[7:0] — incrementado a cada falha detectada

- **Lógica de Funcionamento**

O módulo opera na borda de subida do clock. No reset, todos os bits de validade e os contadores são zerados. A cada ciclo com read=1, o endereço é dividido em tag (bits [4:3]) e index (bits [2:0]). Se valid[index]=1 e tag\_array[index]=tag, ocorre hit: cache\_data recebe data\_array[index] e hit\_count é incrementado. Caso contrário, ocorre miss: o bloco é substituído com os novos dados e miss\_count é incrementado. Os contadores são atualizados exclusivamente quando read=1.

- Sequência de Teste e Resultados

Tabela 2 — Sequência de teste

| Acesso | Endereço | Dado (memória) |
|--------|----------|----------------|
| 1      | 10110    | 8'hA1          |
| 2      | 11010    | 8'hB2          |
| 3      | 10110    | 8'hA1          |
| 4      | 11010    | 8'hB2          |
| 5      | 10000    | 8'hC3          |
| 6      | 00011    | 8'hD4          |
| 7      | 10000    | 8'hC3          |
| 8      | 10010    | 8'hE5          |
| 9      | 10000    | 8'hC3          |

- Log da Simulação

|          |  |            |  |         |  |           |  |                 |  |              |  |         |  |         |
|----------|--|------------|--|---------|--|-----------|--|-----------------|--|--------------|--|---------|--|---------|
| Acesso 1 |  | End: 10110 |  | Tag: 10 |  | Idx: 110  |  | MISS            |  | Dado: a1     |  | Hits: 0 |  | Miss: 1 |
| Acesso 2 |  | End: 11010 |  | Tag: 11 |  | Idx: 010  |  | MISS            |  | Dado: b2     |  | Hits: 0 |  | Miss: 2 |
| Acesso 3 |  | End: 10110 |  | Tag: 10 |  | Idx: 110  |  | HIT             |  | Dado: a1     |  | Hits: 1 |  | Miss: 2 |
| Acesso 4 |  | End: 11010 |  | Tag: 11 |  | Idx: 010  |  | HIT             |  | Dado: b2     |  | Hits: 2 |  | Miss: 2 |
| Acesso 5 |  | End: 10000 |  | Tag: 10 |  | Idx: 000  |  | MISS            |  | Dado: c3     |  | Hits: 2 |  | Miss: 3 |
| Acesso 6 |  | End: 00011 |  | Tag: 00 |  | Idx: 011  |  | MISS            |  | Dado: d4     |  | Hits: 2 |  | Miss: 4 |
| Acesso 7 |  | End: 10000 |  | Tag: 10 |  | Idx: 000  |  | HIT             |  | Dado: c3     |  | Hits: 3 |  | Miss: 4 |
| Acesso 8 |  | End: 10010 |  | Tag: 10 |  | Idx: 010  |  | MISS            |  | Dado: e5     |  | Hits: 3 |  | Miss: 5 |
| Acesso 9 |  | End: 10000 |  | Tag: 10 |  | Idx: 000  |  | HIT             |  | Dado: c3     |  | Hits: 4 |  | Miss: 5 |
| Total:   |  | 9 acessos  |  | Hits: 4 |  | Misses: 5 |  | Taxa de acerto: |  | 4/9 = 44,44% |  |         |  |         |

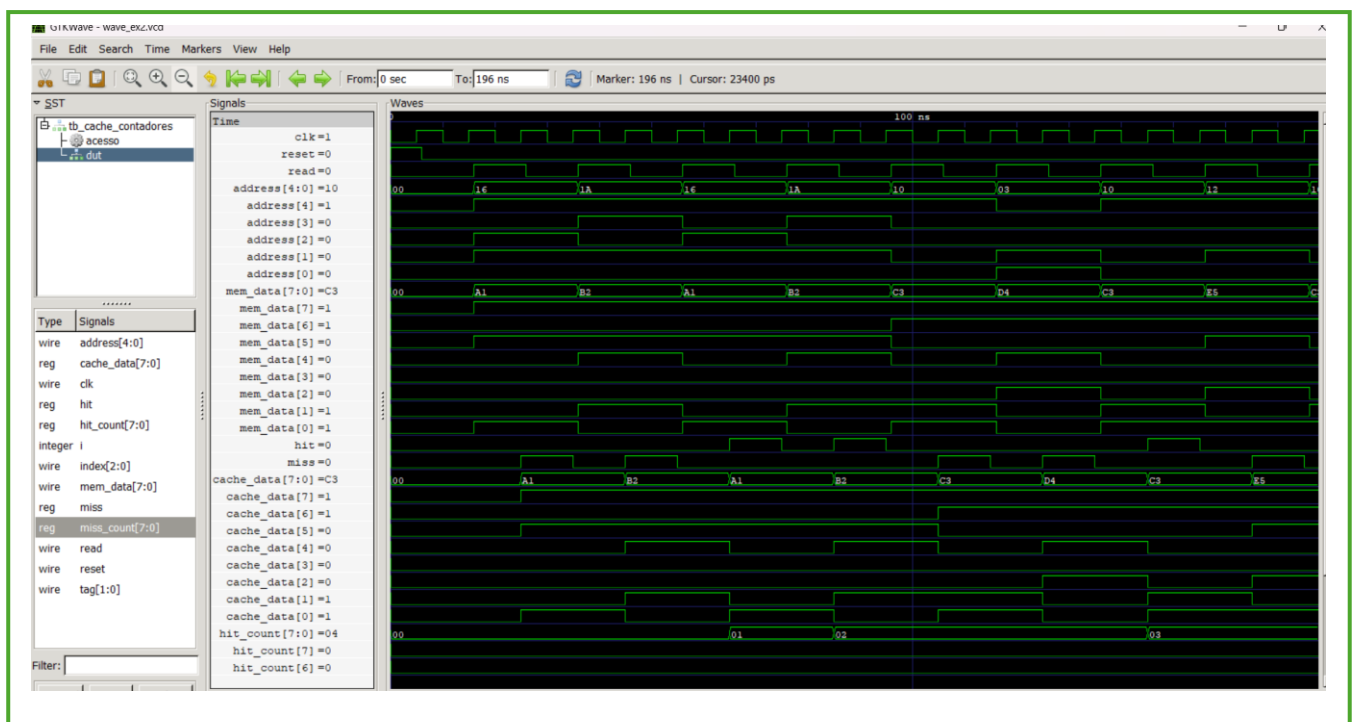
- Resultados Esperados Vs Obtido

**Tabela 3 — Resultados detalhados**

| Ac. | Endereço | Tag | Índice | Res. | Justificativa                                    |
|-----|----------|-----|--------|------|--------------------------------------------------|
| 1   | 10110    | 10  | 110    | MISS | Primeiro acesso — cache vazia (valid=0)          |
| 2   | 11010    | 11  | 010    | MISS | Índice 010 vazio — carrega B2                    |
| 3   | 10110    | 10  | 110    | HIT  | Tag 10 já na posição 110                         |
| 4   | 11010    | 11  | 010    | HIT  | Tag 11 já na posição 010                         |
| 5   | 10000    | 10  | 000    | MISS | Índice 000 vazio — carrega C3                    |
| 6   | 00011    | 00  | 011    | MISS | Índice 011 vazio — carrega D4                    |
| 7   | 10000    | 10  | 000    | HIT  | Tag 10 ainda na posição 000                      |
| 8   | 10010    | 10  | 010    | MISS | Conflito: tag 11→10 no índice 010 (substitui B2) |
| 9   | 10000    | 10  | 000    | HIT  | Tag 10 ainda na posição 000                      |

- Captura de Tela - GTKWave

A Figura 1 apresenta a captura de tela da simulação no GTKWave do Exercício 2, confirmando visualmente todos os resultados obtidos. É possível observar os sinais de clk, reset, read, address[4:0], mem\_data[7:0], hit, miss, cache\_data[7:0], hit\_count[7:0] e miss\_count[7:0] ao longo dos 196 ns de simulação



Na figura, o sinal `address[4:0]` exibe os endereços em hexadecimal: 16 (=10110), 1A (=11010), 10 (=10000), 03 (=00011) e 12 (=10010). O sinal `cache_data` confirma os valores retornados: A1, B2, C3, D4 e E5 conforme cada acesso. O sinal `hit_count` acumula progressivamente os acertos (01, 02, 03, 04) nos acessos 3, 4, 7 e 9, enquanto miss ocorre nos acessos 1, 2, 5, 6 e 8, visíveis pelos pulsos do sinal `miss`.

## Respostas às Perguntas

### 1. Quantos bits do endereço são usados como tag?

2 bits — os bits mais significativos [4:3]. Com endereço de 5 bits e 8 blocos ( $\log_2(8) = 3$  bits de índice), sobram 2 bits para a tag. Exemplo: endereço 10110 → tag = 10.

### 2. Quantos bits são usados como índice?

3 bits — os bits menos significativos [2:0]. Eles determinam em qual das 8 posições da cache o bloco será armazenado. Exemplo: endereço 10110 → índice = 110 (posição 6 da cache).

### 3. Por que os endereços 11010 e 10010 são mapeados para a mesma posição?

Porque os 3 bits menos significativos (índice) são idênticos: 11010 → índice=010 e 10010 → índice=010. Na cache mapeada diretamente, o índice determina exclusivamente a posição, independentemente da tag. As tags são diferentes (11 ≠ 10), por isso ocorre substituição no acesso 8.

### 4. Em qual acesso ocorre substituição?

No acesso 8 (endereço 10010, tag=10, índice=010). A posição 010 estava ocupada com o dado do endereço 11010 (tag=11, dado=0xB2, carregado no acesso 2). A nova entrada com tag=10 substitui o conteúdo anterior.

### 5. Qual era o dado armazenado antes da substituição?

0xB2 — carregado no acesso 2 para o endereço 11010 (tag=11, índice=010). Após a substituição no acesso 8, esse dado é perdido da cache e a posição 010 passa a armazenar 0xE5 (endereço 10010).

### 6. Qual foi a taxa de acerto da cache?

4 hits em 9 acessos =  $4/9 \approx 44,44\%$ . Hits nos acessos 3, 4, 7 e 9. Misses nos acessos 1, 2, 5, 6 e 8.

### 7. O que aconteceria se o número de blocos aumentasse para 16?

Com 16 blocos, seriam necessários 4 bits de índice ( $\log_2(16)=4$ ) e apenas 1 bit de tag (endereço de 5 bits). A nova divisão seria TAG[4] | ÍNDICE[3:0]. Os endereços 11010 (índice=1010, tag=1) e 10010 (índice=0010, tag=1) passariam

a ter índices DIFERENTES, eliminando o conflito do acesso 8. A taxa de acerto aumentaria para  $5/9 \approx 55,56\%$ , pois o miss do acesso 8 se tornaria hit.

#### **8. Qual é a função do bit de validade?**

Indica se a posição da cache contém um dado válido. Após o reset, todos os bits são 0 (inválido). Sem o bit de validade, a cache poderia interpretar dados residuais na memória SRAM como hits legítimos, retornando valores incorretos. Somente após o primeiro miss e carregamento de um bloco é que `valid[index]` é setado para 1, autorizando futuras comparações de tag.

#### **9. Por que o primeiro acesso a um endereço sempre tende a gerar miss?**

Porque ou a posição ainda não foi preenchida (`valid=0`) ou foi preenchida com dado de outro endereço (tag diferente). Sem histórico de acesso prévio àquela posição com aquela tag, a cache não possui o bloco solicitado e precisa buscá-lo na memória principal.

#### **10. Como a localidade temporal aparece nessa sequência de acessos?**

Localidade temporal significa reutilizar dados acessados recentemente. Exemplos na sequência: acessos 1→3 (10110): miss inicial, depois hit imediato — dado reutilizado logo em seguida. Acessos 2→4 (11010): mesmo padrão de miss seguido de hit. Acessos 5→7 (10000): miss e depois hit. Acesso 9 (10000): hit após recarga no acesso 7. A cache aproveita essa localidade mantendo os blocos mais recentes disponíveis, evitando acessos repetidos à memória principal para os mesmos dados.